MindAnchor

Authentication Options

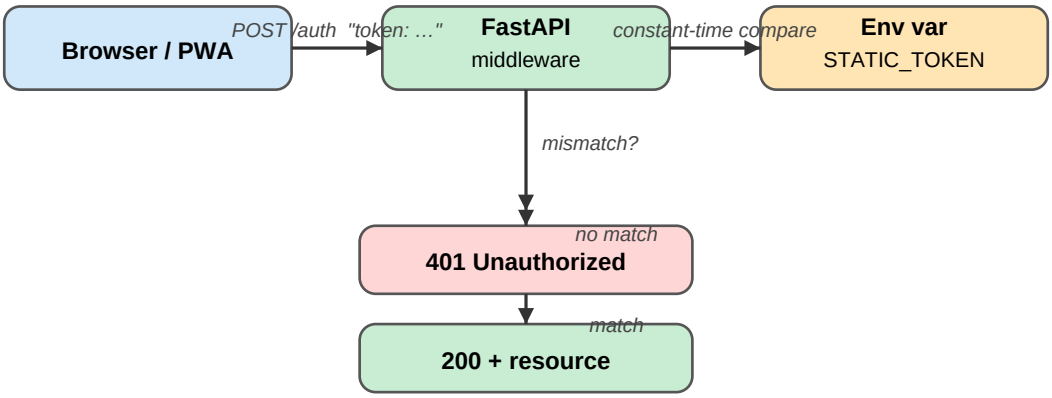
This document compares three authentication strategies for a single-user FastAPI + React PWA. Each option includes a flow diagram and a pros / cons summary.

Option	Approach	Complexity	Recommended?
A	Static Bearer Token	Minimal	Simple setups only
B	Password + JWT	Low–Medium	✔ YES — recommended
C	Cloudflare Access	Medium	Great if on CF stack

Option A — Static Bearer Token

The client sends a long, secret random string in every request header. The server does a constant-time string comparison against an env var. No database, no cryptography library, no sessions.

Flow diagram



Pros & Cons

Pros	Cons
✔ Zero setup — no user table, no crypto library	✗ No expiry — token is valid forever until manually rotated
✔ Token is just a long random string (e.g. openssl rand -hex 32)	✗ Rotation requires updating the env var and redeploying
✔ Easy to understand and audit	✗ No session concept — can't log out without changing the token

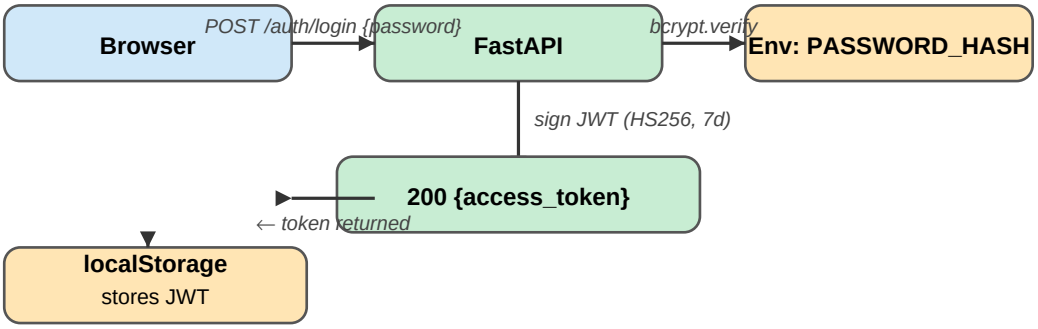
Implementation sketch: Set `STATIC_TOKEN` in your env. Add a FastAPI dependency that reads `Authorization: Bearer <token>` and calls `secrets.compare_digest()`. Return 401 on mismatch.

Option B — Password Login + JWT

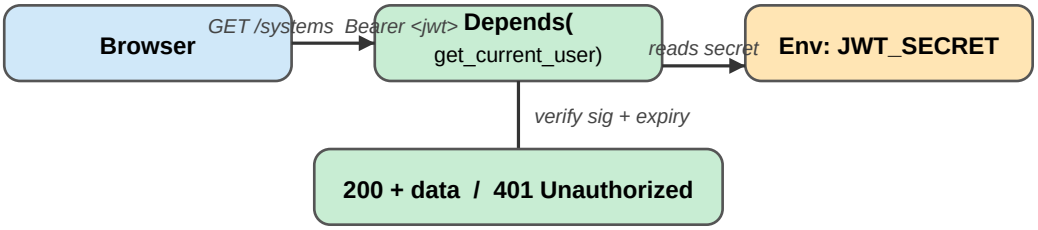
RECOMMENDED for MindAnchor. The user logs in once with a password (stored only as a bcrypt hash in an env var — no DB row needed). The server issues a short-lived JWT. Every subsequent request carries that JWT; the server verifies the signature and expiry without any DB look-up.

Flow diagram

Step 1 — Login



Step 2 — Every protected request



Pros & Cons

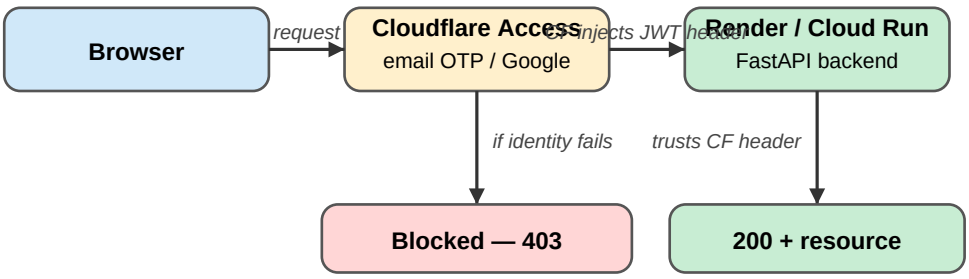
Pros	Cons
✔ Industry standard (OAuth2 Bearer / RFC 7519) ✔ Expiry built in — 7-day tokens, then re-login ✔ Rotating JWT_SECRET instantly invalidates all sessions ✔ No user table — password hash stored only in env var	✗ Slightly more code than Option A (~50 lines with python-jose + passlib) ✗ Token stored in localStorage — readable by JS (mitigated by HTTPS + no 3rd-party scripts) ✗ Stateless — can't revoke a single token before expiry without a denylist

Required env vars: `PASSWORD_HASH` (bcrypt, generate with `passlib.hash.bcrypt.hash('yourpassword')`), `JWT_SECRET` (random 32-byte hex), `JWT_EXPIRE_DAYS` (default 7). **Required packages:** `python-jose[cryptography]`, `passlib[bcrypt]`.

Option C — External Proxy Auth (Cloudflare Access)

Authentication is delegated entirely to Cloudflare Access. The browser must pass Cloudflare's identity check (email OTP, Google, GitHub, etc.) before any traffic reaches your backend. Cloudflare injects a signed JWT header (`Cf-Access-Jwt-Assertion`) which FastAPI can optionally verify.

Flow diagram



** FastAPI reads the Cf-Access-Jwt-Assertion header (already verified by Cloudflare)*

Pros & Cons

Pros	Cons
✔ Zero auth code in your app — CF handles 100% of it	✗ Ties your stack to Cloudflare — vendor lock-in
✔ MFA / phishing-resistant login (passkeys, hardware keys) for free	✗ Doesn't work cleanly with Vercel PWA's /api rewrite proxy (both CF and Vercel handle routing)
✔ Blocks at network edge — malicious traffic never hits your server	✗ More infrastructure to configure and maintain
✔ Audit log, IP rules, and device posture checks in one place	✗ Overkill for a private single-user app not exposed to the internet

When to choose this: If your PWA is served via Cloudflare (not Vercel) and you want zero-effort MFA without writing a single line of auth code. For MindAnchor's current Vercel + Cloud Run setup, Option B is simpler.